


VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

**School of Computer Science Engineering and Information Systems**
**B.Tech Information Technology**
**BCSE355L : Cloud Architecture Design**
**FALL 2026-2027**
**Course Project -Final Review**
**(Review Date : 16/10/2026 )**
**Link to Upload: https://forms.gle/cyQMu6huDLiHFaZV8**

| |
|---|
| TEAM NAME : xyz |
| Team Member(s) with Reg # and Name :<br>Tanmayananda Sahoo ; 24BIT0011<br>Shinde Atharav Vikas ; 24BIT0024<br>Bhadange Vedant Rajkumar ; 24BIT0504 |
| Project Title : Kubernetes Resource Right-Sizing Advisor |
| **1. Introduction**<br>1.1 Background<br>1.2 Problem Statement<br>1.3 Literature Survey (10 per student , only from reputed TR - IF journals) |
| **2. Overview and Planning**<br>2.1 Proposed System Overview<br>2.2 Challenges<br>2.3 Assumptions<br>2.4 Architecture Specifications<br>2.5 Hardware Requirements<br>2.6 Software Requirements |
| **3. System Design**<br>3.1 High-Level Design<br>3.2 Low-Level Design |
| **4. Implementation : 100%** |
| Any other related information, you want to add. |
| For Evaluation only<br>Status : ACCEPTED / REJECTED |
| |

| | |
|---|---|
| VIT® Vellore Institute of Technology (Deemed to be University under section 3 of UGC Act, 1956) | **School of Computer Science Engineering and Information Systems**<br>**B.Tech Information Technology**<br>**BCSE355L : Cloud Architecture Design**<br>**FALL 2026-2027**<br>**Course Project -Final Review** |
| **Project Title:** | Kubernetes Resource Right-Sizing Advisor |
| **Team Name** | **xyz** |

**Project Team**

| S.No | Register Number | Student Name | Signature | Guided By |
|---|---|---|---|---|
| 1 | 24BIT0011 | Tanmayananda Sahoo | | **Nadesh R K** |
| 2 | 24BIT0024 | Shinde Atharav Vikas | | **Nadesh R K** |
| 3 | 24BIT0504 | Bhadange Vedant Rajkumar | | **Nadesh R K** |

**Team Member(s) Contribution and Performance Assessment**

| Components | Student 1 | Student 2 | Student 3 |
|---|---|---|---|
| Literature Survey (03) | | | |
| Design (06) | | | |
| Implementation (06) | | | |
| Total (15) | | | |

| Expectation for Next Reviews | Comments |
|---|---|
| | |

**Name & Signature of the Evaluator**

# 1. Introduction

## 1.1 Background

Kubernetes is now the usual way to run containerized applications. When we deploy an application, one of the first things we have to do is tell Kubernetes how much CPU and memory the pod will need. This is called a resource request. Kubernetes uses it to decide where to place the pod. In most billing models it is also the amount that gets charged, and not what the pod really uses after it starts running. While reading about this topic we saw that the gap between what is asked for and what is used is one of the most common reasons for wasted cloud money.

Surveys say the same thing. Flexera's State of the Cloud reports have put wasted cloud spend at around 30 percent of the total, year after year. A recent Frontiers article on FinOps says that almost one third of all cloud spending goes to infrastructure that is not used well (Everman et al., 2021, as cited in the Eco-FinOps study).

This happens for a simple reason. When a developer is not sure how much an application needs, the safe choice is to ask for more. If the request is too low, the pod can be slowed down, or killed when it runs out of memory. Nobody wants that, so the numbers go up and almost never come down, even after months of running in production. Developers also do not have an easy way to see how their requests compare with the real usage over time.

## 1.2 Problem Statement

There is no simple built-in way in a normal Kubernetes cluster to answer this question: how much of what we requested are we really using, and what should we request instead? Tools like the Vertical Pod Autoscaler exist, but they are not switched on by default. They change the values automatically, so the team gets no chance to review the numbers, and they do not give a cost figure. Without a cost figure it is hard to explain the change to people outside the engineering team.

This project builds a smaller and more open version of that idea. The tool watches how much CPU and memory a set of pods really uses over time and compares it with what each pod requested. It calculates a better value using a simple method, which is the 90th percentile of the past usage plus a safety margin. Then it changes the waste into an approximate monthly dollar figure using published cloud prices. We kept every step visible and easy to explain, and we did not use machine learning, because the project is as much about showing the reasoning as it is about the final number.

## 1.3 Literature Survey

Each team member read ten papers on one part of the problem. Tanmayananda read about resource management and autoscaling in Kubernetes. Shinde read about cloud cost optimization and FinOps. Vedant read about monitoring and the statistical methods used for capacity planning. After some of the entries we added a short note on what we took from the paper.

### Tanmayananda Sahoo (24BIT0011): Kubernetes Resource Management and Autoscaling

[1] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in Proc. 10th European Conf. on Computer Systems (EuroSys), ACM, 2015. Note: Kubernetes is mostly based on this system. The paper says that even a few percent of better utilization saves Google millions of dollars. Our project uses the same idea on a much smaller scale.

[2] D. Balla, C. Simon, and M. Maliosz, “Adaptive scaling of Kubernetes pods,” in Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS), 2020, pp. 1–5. Note: it covers scaling pods both horizontally and vertically, and helped us see why VPA and HPA are usually talked about together.

[3] Y. Al-Dhuraibi, F. Paraiso, N. Djarallah, and P. Merle, “Autonomic vertical elasticity of Docker containers with ElasticDocker,” in Proc. IEEE 10th Int. Conf. on Cloud Computing (CLOUD), 2017, pp. 472–479. Note: an early try at resizing containers automatically, from before the Kubernetes VPA matured. It explains the problems of resizing without a restart, which is why we did not resize our pods during a test.

[4] J. Santos, T. Wauters, B. Volckaert, and F. De Turck, “gym-hpa: Efficient auto-scaling via reinforcement learning for complex microservice-based applications in Kubernetes,” in Proc. IEEE/IFIP NOMS, 2023, pp. 1–9.

[5] B. Liu, R. Buyya, and A. Huang, “CoPa: A combined autoscaling method for Kubernetes,” in Proc. IEEE Int. Conf. on Web Services (ICWS), 2021, pp. 416–425.

[6] J. Tong, M. Wei, M. Pan, and Y. Yu, “A holistic auto-scaling algorithm for multi-service applications based on balanced queuing network,” in Proc. IEEE ICWS, 2021, pp. 531–540.

[7] P. Kang and P. Lama, “Robust resource scaling of containerized microservices with probabilistic machine learning,” in Proc. IEEE/ACM 13th Int. Conf. on Utility and Cloud Computing (UCC), 2020, pp. 122–131.

[8] N. S. Joshi, R. Raghuwanshi, Y. M. Agarwal, B. Annappa, and D. Sachin, “ARIMA-PID: container auto scaling based on predictive analysis and control theory,” Multimedia Tools and Applications, 2023.

[9] S. Y. Noh et al., “PVA: the persistent volume autoscaler for stateful applications in Kubernetes,” IEEE Access, vol. 12, pp. 179130–179143, 2024. Note: it applies the same right-sizing idea to storage volumes instead of CPU and memory. This is outside our scope, but it is a good direction for future work.

[10] L. Baresi, D. Y. X. Hu, G. Quattrocchi, and L. Terracciano, “KOSMOS: Vertical and horizontal resource autoscaling for Kubernetes,” in Proc. Int. Conf. on Service-Oriented Computing (ICSOC), Springer, 2021, pp. 821–829.

### Shinde Atharav Vikas (24BIT0024): Cloud Cost Optimization and FinOps

[11] P. Nawrocki and M. Smendowski, “FinOps-driven optimization of cloud resource usage for high-performance computing using machine learning,” Journal of Computational Science, vol. 79, art. 102292, 2024. Note: it talks about the balance between over-provisioning and the risk of missing performance targets. This is the same balance that made us choose a percentile and not the maximum value.

[12] P. Nawrocki and M. Smendowski, “A survey of cloud resource consumption optimization methods,” Journal of Grid Computing, vol. 23, no. 1, art. 5, 2025.

[13] B. Śnieżyński, P. Nawrocki, M. Wilk, M. Jarząb, and K. Zieliński, “VM reservation plan adaptation using machine learning in cloud computing,” Journal of Grid Computing, vol. 17, pp. 797–812, 2019.

[14] P. Nawrocki, P. Osypanka, and B. Posłuszny, “Data-driven adaptive prediction of cloud resource usage,” Journal of Grid Computing, vol. 21, no. 1, art. 6, 2023.

[15] W. Sus and P. Nawrocki, “Signature-based adaptive cloud resource usage prediction using machine learning and anomaly detection,” Journal of Grid Computing, vol. 22, no. 2, art. 46, 2024. Note: the authors say that unusual one-time spikes in the usage history lead to poor provisioning decisions later. This made us careful with the 6 hour retention in Prometheus. It is long enough to smooth out noise and short enough to repeat in a demo.

[16] C. Bala Subramanian, S. T. Bharathi, and S. Shanmugapriya, “Achieving cloud resource optimization with trust-based access control: a novel ML strategy for enhanced performance,” MethodsX, 2025, doi: 10.1016/j.mex.2025.103461.

[17] D. Fernández-Cerero, A. Fernández-Montes, A. Jakóbik, J. Kołodziej, and M. Toro, “SCORE: Simulator for cloud optimization of resources and energy consumption,” Simulation Modelling Practice and Theory, vol. 82, pp. 160–173, 2018.

[18] M. Yadav and A. Mishra, “An enhanced ordinal optimization with lower scheduling overhead based novel approach for task scheduling in cloud computing environment,” Journal of Cloud Computing, vol. 12, no. 1, art. 8, 2023.

[19] D. F. Kirchoff, V. Meyer, R. N. Calheiros, and C. A. De Rose, “Evaluating machine learning prediction techniques and their impact on proactive resource provisioning for cloud environments,” The Journal of Supercomputing, vol. 80, no. 15, pp. 21920–21951, 2024.

[20] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource Central: understanding and predicting workloads for improved resource management in large cloud platforms,” in Proc. 26th Symp. on Operating Systems Principles (SOSP), ACM, 2017, pp. 153–167. Note: a Microsoft Azure paper about predicting workload behavior at large scale. It says that past usage is a fairly good guide for near-future usage. Our 15 minute lookback depends on the same assumption, only on a very small scale.

### Bhadange Vedant Rajkumar (24BIT0504): Monitoring, Observability, and Statistical Methods

[21] J. Dean and L. A. Barroso, “The tail at scale,” Communications of the ACM, vol. 56, no. 2, pp. 74–80, 2013. Note: this paper made us choose the 90th percentile and not the average. The authors say that averages hide the behavior that really matters. The paper is about response time and not about CPU or memory, but the same argument works for resource usage.

[22] A. Podzimek, L. Bulej, L. Y. Chen, W. Binder, and P. Tůma, “Analyzing the impact of CPU pinning and partial CPU loads on performance and energy efficiency,” in Proc. IEEE/ACM 15th Int. Symp. on Cluster, Cloud and Grid Computing (CCGrid), 2015, pp. 1–10. Note: useful for our stress-ng setup. Their measurements of partial CPU loads helped us understand why the load we measured was a bit different from the load we asked for.

[23] C. Reiss, J. Wilkes, and J. L. Hellerstein, “Heterogeneity and dynamicity of clouds at scale: Google trace analysis,” in Proc. 3rd ACM Symp. on Cloud Computing (SoCC), 2012.

[24] M. G. Xavier, C. H. C. Cano, V. Meyer, and C. A. F. De Rose, “INTP: quantifying cross-application interference via system-level instrumentation,” in Proc. IEEE 34th Int. Symp. on Computer Architecture and High Performance Computing (SBAC-PAD), 2022, pp. 231–240.

[25] E. Casalicchio and V. Perciballi, “Auto-scaling of containers: the impact of relative and absolute metrics,” in Proc. IEEE 2nd Int. Workshops on Foundations and Applications of Self* Systems (FAS*W), 2017, pp. 207–214, doi: 10.1109/FAS-W.2017.149. Note: they found that absolute metrics scale containers more accurately than relative (percentage) ones for CPU heavy workloads. This is one reason we show millicore values and percentages together in our report.

[26] E. Casalicchio and V. Perciballi, “Measuring Docker performance: what a mess!!!,” in Proc. 8th ACM/SPEC Int. Conf. on Performance Engineering Companion (ICPE), 2017, pp. 11–16.

[27] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” ACM Queue, vol. 14, pp. 70–93, 2016.

[28] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, “An updated performance comparison of virtual machines and Linux containers,” in Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS), 2015, pp. 171–172.

[29] Z. Kozhirbayev and R. O. Sinnott, “A performance comparison of container-based technologies for the cloud,” Future Generation Computer Systems, vol. 68, pp. 175–182, 2017.

[30] E. Casalicchio, “Autonomic orchestration of containers: problem definition and research challenges,” in Proc. 10th EAI Int. Conf. on Performance Evaluation Methodologies and Tools (VALUETOOLS), EAI, 2016.

Taken together, these papers point the same way. Use percentiles and not averages (Dean and Barroso). Keep the requested and the actual usage visible side by side (Casalicchio and Perciballi). In the FinOps papers the main problem is not the math, it is that nobody looks at the numbers. Our project is a small attempt to close that gap for one Kubernetes namespace.

# 2. Overview and Planning

## 2.1 Proposed System Overview

The system, which we call RightSize, has seven parts. All of it runs on one Kubernetes cluster on a laptop, except Amazon CloudWatch, which runs on AWS.

1. Four sample applications are deployed with requests that are set higher than what they use. This way the waste is a known amount, and we can check our results against it.
2. Prometheus and kube-state-metrics collect what each pod requested and what it really uses, every 15 seconds.
3. A set of Python scripts reads this data from Prometheus.
4. The recommendation engine calculates the 90th percentile of the usage and adds 20 percent. This is the recommended value.
5. The cost module changes the gap between the requested and the recommended value into a monthly dollar figure, using the AWS Fargate price list.
6. The results are shown on a Grafana dashboard, through a Pushgateway, and on a dashboard page that we built ourselves.
7. The results are also sent to Amazon CloudWatch as custom metrics. We built a CloudWatch dashboard and an alarm on them.

The only cloud service we use is CloudWatch. The prices in the cost calculation are fixed numbers taken from the public AWS Fargate price page, so we do not connect to any AWS billing service.

## 2.2 Challenges

Some things did not work the way we expected at first. We write them here because they changed some of our later decisions.

- Making a known amount of waste was harder than we thought. First we used the --vm option of stress-ng for memory. We did not know that it keeps writing to memory all the time, and that the --cpu-load setting only slows down the --cpu part. So some pods used more CPU than their limit and got throttled, and our first results looked almost the same as the limit. We fixed it by adding --vm-hang, so the memory part allocates once and then stays idle.
- Pod names in Kubernetes are not fixed. A deployment called app-1 runs as a pod like app-1-79447bc758-h9cvw, because Kubernetes adds a hash and a random ending. Our first script used the short names and found nothing in Prometheus. We changed it to read the pod names from Prometheus.
- Setting up the RBAC permissions so that Prometheus could reach the kubelet for cAdvisor took a few tries. A missing permission only shows a 403 error and does not say which permission is missing.

- After the cluster was left running for a few days, three of the four CPU stressors stopped by themselves while the pods still showed Running. The tool then reported 0 CPU usage for them. Restarting the pods fixed it.
- CloudWatch has a free tier of 10 custom metrics, and every different name and dimension counts as one metric. We had to keep the number of metrics small. We also used the deployment name (app-1) and not the pod name, because the pod name changes after every restart and each new name would be a new paid metric.
- Everything runs on one laptop with about 7 GB of RAM. We kept the number of pods small, so the project cannot say much about a big cluster with many nodes.

## 2.3 Assumptions

- The usage of the four sample pods is steady while we observe it. We did not test a workload that changes over time, like a daily traffic pattern. So the 90th percentile is tested on fairly steady data and not on the bursty traffic it is meant for.
- Each pod has the same request and limit (the Guaranteed QoS class). This avoids scheduling effects, but real deployments are not always set up this way.
- The AWS Fargate price per vCPU and per GB is a good stand-in for the cost of reserved capacity. A real bill on a shared cluster, where many pods sit on the same machine, would be different.
- One node is enough for this course project. We do not look at bin-packing or scheduling across nodes.
- The AWS account has no other custom CloudWatch metrics, because the free tier of 10 metrics is shared with them.
- The results reach CloudWatch only when main.py runs. We assume it is run by hand, or in a loop every few minutes.

## 2.4 Architecture Specifications

The cluster has three namespaces. rightsizing-demo has the four sample workloads. monitoring has Prometheus, the Pushgateway and Grafana. kube-system has kube-state-metrics, next to the normal Kubernetes components. The data moves in one direction, as shown in Figure 1.




*Figure 1: Data flow through the RightSize pipeline*

Prometheus has four scrape jobs. The first uses Kubernetes service discovery to find every node and reads the cAdvisor metrics through the kubelet. This gives the real CPU and memory usage. The second reads kube-state-metrics for the requested values. The third is Prometheus watching itself. The fourth reads the Pushgateway,

where our Python program puts its results. A separate ServiceAccount, ClusterRole and ClusterRoleBinding give Prometheus read-only access to only what it needs.

The Python program runs once and then stops, so Prometheus cannot collect from it directly. The Pushgateway holds the results until Prometheus reads them, and this is how the results reach Grafana.

For CloudWatch, the program sends 9 custom metrics to the namespace RightSize. They are listed in the table below. A CloudWatch dashboard with 4 charts and one alarm are built on them. The alarm watches TotalWastedCostUSD and goes off when it is above 30 dollars per month. The program uses a separate IAM user that is only allowed to send metrics (PutMetricData) to the RightSize namespace. The access keys are kept in environment variables, outside the project folder.

| **Metric** | **Dimension** | **Count** |
|---|---|---|
| TotalWastedCostUSD, TotalRequestedCostUSD, TotalRecommendedCostUSD | none | 3 |
| WastedCostUSD | Deployment (app-1 to app-4) | 4 |
| AvgWastePercent | Resource (cpu, memory) | 2 |
| Total | | 9 |

Figure 2 shows the system from the user's side. The DevOps engineer deploys the sample workloads, runs the pipeline and looks at the three dashboards. Running the pipeline includes six smaller steps. These are collecting the metrics, computing the recommendation, estimating the cost, writing the CSV and JSON report, publishing the results to the Pushgateway and publishing the metrics to CloudWatch. Prometheus and CloudWatch are drawn outside the system because they are separate tools. Prometheus supplies the usage data and reads the Pushgateway. CloudWatch stores the metrics we publish and raises the alarm.

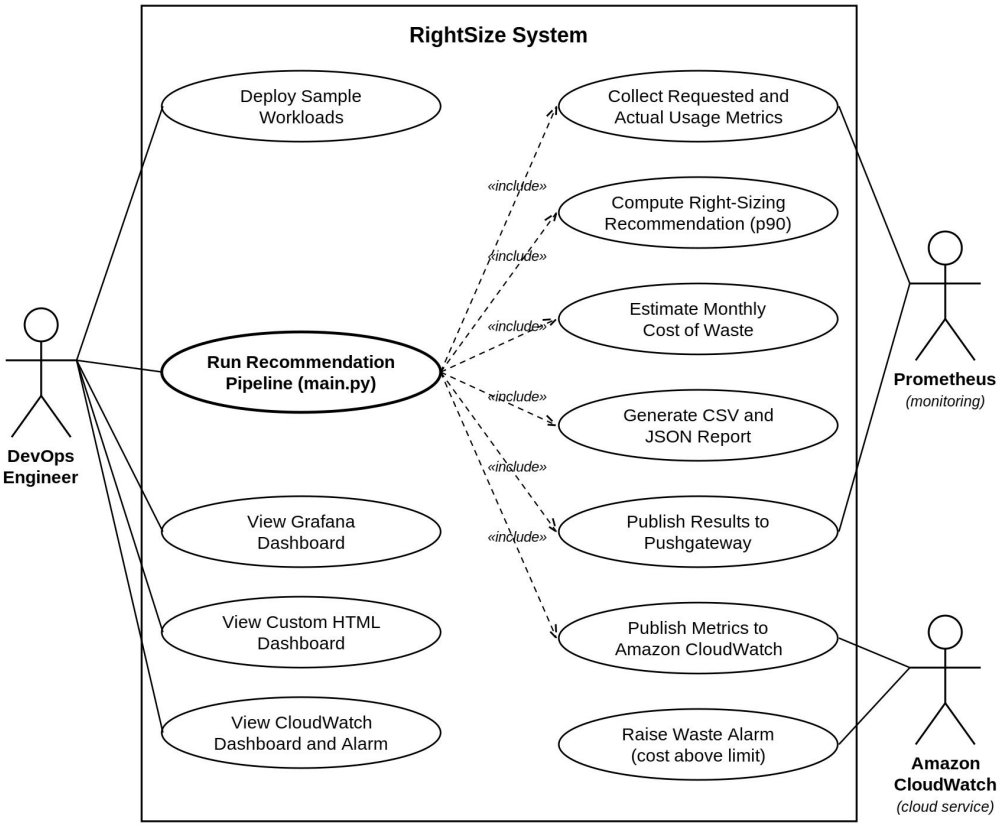



*Figure 2: Use case diagram of RightSize*

## 2.5 Hardware Requirements

| Component | Minimum | Used in this project |
|---|---|---|
| RAM | 4 GB free (tight) | About 7 GB in total, enough with some room left |
| CPU | 2 logical cores | A normal laptop CPU (x86_64) |
| Disk space | About 5 GB free | For the container images: kind node, Prometheus, Pushgateway, Grafana, kube-state-metrics, stress-ng |
| Network | Internet connection | To download images and to send metrics to CloudWatch |
| OS | Linux (tested), or macOS or Windows with Docker Desktop | Ubuntu-based Linux |

## 2.6 Software Requirements

| Software | Purpose |
|---|---|
| Docker Engine | Runs the kind cluster and all the pods |
| kind (Kubernetes-in-Docker) | Makes the single-node Kubernetes cluster on the laptop |

| | |
|---|---|
| kubectl | Command line tool for the cluster |
| Prometheus v2.53.0 | Stores the metrics and answers queries |
| Prometheus Pushgateway v1.9.0 | Holds the results of the Python program so Prometheus can read them |
| Grafana v11.1.0 | Dashboard for the metrics and the results |
| kube-state-metrics v2.20.0 | Gives the requested values of the pods as metrics |
| stress-ng (alexeiled/stress-ng image) | Creates a known CPU and memory load in the test pods |
| Python 3 | Runs the recommendation, cost and pipeline code |
| pandas, requests, boto3, pytest | Percentile calculation, queries to Prometheus, sending metrics to AWS, unit tests |
| HTML and JavaScript | Our own dashboard page |
| Amazon CloudWatch | Custom metrics, dashboard and alarm |
| AWS IAM | A separate user that can only send metrics |

# 3. System Design

## 3.1 High-Level Design

The system has five layers. The workload layer is the four sample deployments. The metrics layer is kube-state-metrics and the cAdvisor inside each node, which both give metrics in the Prometheus format. The storage layer is Prometheus, which reads both sources every 15 seconds and keeps 6 hours of data. The analytics layer is a set of Python files that read Prometheus, do the calculations, add the cost and print the report. The last layer is where the results are shown: Grafana, our own HTML dashboard, and Amazon CloudWatch.




*Figure 3: Python modules and how they are used*

main.py is the starting point. It first asks Prometheus what each pod has requested, using an instant query. Then it asks for the usage of each pod over the last 15 minutes, using a range query. Both are given to recommender.py, which returns the recommended value and the waste percent for each pod and resource. cost_estimator.py then adds the monthly dollar figure. The table is printed on the screen and saved as rightsizing_report.csv and rightsizing_report.json. After that, the results are sent to the Pushgateway and to CloudWatch.

## 3.2 Low-Level Design

Here we list the real functions. We felt this was more useful than a UML class diagram, because the project has no class hierarchy, only a short data pipeline.

### recommender.py

```
SAFETY_BUFFER = 0.20   # 20% extra above the 90th percentile
PERCENTILE = 0.90

def compute_p90(usage_samples):
    # returns the 90th percentile of the usage samples

def compute_recommendation(usage_samples, buffer=SAFETY_BUFFER):
    # returns compute_p90(usage_samples) * (1 + buffer)

def compute_waste_pct(requested, recommended):
    # returns (requested - recommended) / requested * 100
    # a negative result means the pod is under-provisioned

def build_report_row(pod_name, resource, requested, usage_samples, buffer=SAFETY_BUFFER):
    # puts the three functions above together into one report row
```

### cost_estimator.py

```
PRICE_PER_VCPU_HOUR = 0.04048   # USD, AWS Fargate on-demand, US East (N. Virginia)
PRICE_PER_GB_HOUR = 0.004445
HOURS_PER_MONTH = 730

def monthly_cost(value, resource):
    # changes millicores or Mi into an estimated monthly USD figure

def add_cost_columns(df):
    # adds requested_cost_usd, recommended_cost_usd, wasted_cost_usd
```

### prometheus_client.py

```
class PrometheusClient:
    def instant_query(self, query):
        # GET /api/v1/query  (one current value, used for requested amounts)

    def range_query(self, query, start, end, step='15s'):
        # GET /api/v1/query_range  (past samples, used for real usage)
```

### cloudwatch_publisher.py

```
NAMESPACE = "RightSize"
DEFAULT_REGION = "ap-south-1"

def deployment_name(pod_name):
    # app-1-79447bc758-h9cvw  ->  app-1

def build_metric_data(df, timestamp=None):
    # makes the list of 9 CloudWatch metrics, no AWS call here

def count_unique_metrics(metric_data):
    # each name + dimension set is one billable metric

def publish_to_cloudwatch(df, region=None, client=None, timestamp=None):
    # sends the metrics with put_metric_data
```

### main.py: the steps

1. Query kube_pod_container_resource_requests for CPU and memory (instant query).
2. Take the list of pods from the labels that Prometheus returns, and not from a fixed list of names.
3. For each pod and resource, query the last 15 minutes of usage. For CPU use rate(container_cpu_usage_seconds_total[2m]). For memory use container_memory_working_set_bytes directly.
4. Give the requested value and the usage samples to build_report_row().
5. Give the result to add_cost_columns().
6. Print the table and save it as rightsizing_report.csv and rightsizing_report.json.
7. Push the results to the Pushgateway with push_metrics().
8. Publish the 9 metrics to CloudWatch with publish_to_cloudwatch().

9. If step 7 or step 8 fails, print a warning and carry on, because the report is already saved.

One thing we should explain is why CPU needs rate() and memory does not. container_cpu_usage_seconds_total only goes up. It is the total CPU time used since the container started, so by itself it does not tell us how busy the pod is right now. rate() over 2 minutes changes it into CPU seconds used per second, which is the real usage. Memory does not have this problem, because container_memory_working_set_bytes already shows the current value.

## 4. Implementation

At this review the implementation is complete. Everything we planned is built, tested and working on the live cluster. This includes the cluster, the sample workloads, the monitoring, the recommendation engine, the cost estimator, the Grafana dashboard, our own HTML dashboard, and the CloudWatch part (custom metrics, dashboard and alarm). The only thing we did not build is an optional idea to apply the recommended values to the cluster automatically with kubectl patch.

We wrote 21 unit tests: 8 for the recommendation code, 4 for the cost code and 9 for the CloudWatch code. The tests use numbers that we worked out by hand, and we ran them before the code was used on live data. The CloudWatch test also checks our request against the official AWS API definition, without sending anything to AWS.

The table below is the real output of main.py on the live cluster, from the run that also sent the metrics to CloudWatch. Pod names are shortened to app-1 to app-4. The real names have a hash and a random ending, for example app-1-66f4448dfc-qwv77.

| Pod | Resource | Requested | p90 actual | Recommended | Waste % | Wasted $/mo |
|---|---|---|---|---|---|---|
| app-1 | cpu | 500 m | 52.69 | 63.23 | 87.4% | $12.91 |
| app-1 | memory | 256 Mi | 58.63 | 70.36 | 72.5% | $0.59 |
| app-2 | cpu | 400 m | 64.22 | 77.07 | 80.7% | $9.54 |
| app-2 | memory | 256 Mi | 86.35 | 103.62 | 59.5% | $0.48 |
| app-3 | cpu | 300 m | 19.80 | 23.76 | 92.1% | $8.17 |
| app-3 | memory | 128 Mi | 31.25 | 37.50 | 70.7% | $0.29 |
| app-4 | cpu | 250 m | 54.07 | 64.89 | 74.0% | $5.47 |
| app-4 | memory | 128 Mi | 106.38 | 127.66 | 0.3% | $0.01 |
| Total | | | | | | $37.46 |

The total wasted spend for the four pods is $37.46 per month. The memory of app-4 is our control case, a pod that is almost correctly sized, and it shows only 0.3 percent waste. In an earlier run its recommended value was a little above the

request, which gave a small negative waste. The formula shows this as a negative number, and it means the pod is a little under-provisioned and not wasting anything.

The CloudWatch part works like this. The 9 metrics reach CloudWatch in the Mumbai region (ap-south-1). The CloudWatch dashboard has four charts: the total wasted spend, the requested against the recommended against the wasted cost, the wasted cost for each app, and the average waste percent for CPU and memory. The alarm is set at 30 dollars per month. Our total is about 37 dollars, which is above this limit, so the alarm shows In alarm. We use 9 custom metrics, 1 dashboard and 1 alarm, which is inside the CloudWatch free tier.

## Any other related information

Three problems from this work are worth mentioning, because they changed parts of our design and we can explain them in more detail if asked.

The first was that our Python script could not find any pods, even though Prometheus clearly had the data (we could see it in the Prometheus web page). We had written pod names like app-1 in the script, but Kubernetes names running pods with a hash and a random ending, so the exact name never matched. We fixed it by taking the pod list from the labels that Prometheus returns. This also means the script will keep working if a deployment is scaled to more than one replica.

The second was that our very first live run showed almost the same waste for all four pods, all close to minus 20 percent, and not the different numbers we had planned. After reading the stress-ng documentation we found that its --cpu-load option only works on the --cpu part and has no effect on the --vm memory part, which by default keeps writing to memory at full speed. This pushed the CPU demand above the pod limit and caused throttling, so the measured usage was stuck at the limit. Adding --vm-hang made the memory part go idle after the first allocation, and this fixed it.

The third was found late, after the cluster had been left running for a few days. The tool reported exactly 0.00 CPU usage for three pods, and the fourth one showed usage close to its limit. The pods had restarted 7 times and showed Running. When we checked the raw Prometheus counter twice, 30 seconds apart, the number had not changed at all, so the stress-ng CPU part had stopped working while the container was still alive. Kubernetes only checks that the main process is running, and not that it is doing useful work. We restarted the pods with kubectl rollout restart and the numbers were normal again. For a long demo we should restart the pods first, and a liveness check would be a good fix later.

We wrote these down because we think the debugging is as useful as the final numbers. A project that shows only clean results, with no problems on the way, usually has not been run for real.